END-TO-END DEVOPS AUTOMATION

Complete Step-by-Step Implementation Guide

Git · Terraform · Ansible · Jenkins · Docker · Kubernetes

AWS EC2 · Ubuntu 22.04 · CI/CD Pipeline · Kubernetes Cluster

Includes actual Terraform code used to provision devops-master & devops-agent

Project Overview

What You Are Building

A full production-style DevOps pipeline from scratch. A web application is containerized with Docker, automatically built by Jenkins from Git, infrastructure is provisioned by Terraform, machines are configured by Ansible, and the app is deployed on Kubernetes — all connected end-to-end.

Infrastructure — 4 Machines Total

Machine	Role	Size	How Created
`devops-master`	Ansible Master + Jenkins + Terraform	t3.medium	Terraform (My tf files)
`devops-agent`	Jenkins Agent — Docker enabled	t3.medium	Terraform (My tf files)
`k8s-control-plane`	Kubernetes Master Node	t3.medium	Terraform (Phase 6)
`k8s-worker-1`	Kubernetes Worker Node	t3.medium	Terraform (Phase 6)

> **NOTE:** devops-master and devops-agent are provisioned using YOUR Terraform files (data.tf, locals.tf, main.tf, provider.tf). The K8s machines are provisioned later using a separate Terraform config in Phase 6.

Tools and Their Purpose

Tool	Installed On	What It Does
`Git`	devops-master	Stores all code, configs, playbooks, manifests in one repo
`Terraform`	devops-master	Provisions EC2 instances on AWS automatically
`Ansible`	devops-master	Installs and configures Jenkins, Docker, Kubernetes on all machines
`Jenkins`	devops-master	CI/CD — pulls code, builds Docker image, runs tests automatically
`Docker`	devops-agent	Packages the NGINX app into a container image
`Kubernetes`	k8s-control-plane + worker	Deploys and manages the containerised app with 2 replicas

Git Repository Structure

Everything lives in one Git repository. All automation is driven from this repo:

```
devops-project/
├── app/
│   └── index.html                  # The web page served by NGINX
├── Dockerfile                      # Builds the NGINX container image
├── Jenkinsfile                     # CI/CD pipeline — 4 stages
├── terraform-phase1/               # YOUR Terraform files (devops-master + agent)
│   ├── data.tf
│   ├── locals.tf
│   ├── main.tf
│   └── provider.tf
├── terraform-phase2/               # Terraform for K8s machines
│   └── main.tf
├── ansible/
│   ├── site.yml                    # Master playbook — runs all
│   ├── inventory/
│   │   └── hosts.ini               # Machine IPs and SSH settings
│   └── playbooks/
│       ├── setup_master.yml        # Jenkins + Terraform + Java on master
│       ├── setup_agent.yml         # Docker + Java on agent
│       ├── configure_jenkins_node.yml  # Register agent with Jenkins
│       └── setup_kubernetes.yml    # Full K8s cluster setup
└── kubernetes/
    ├── deployment.yml              # K8s Deployment — 2 replicas
    └── service.yml                 # NodePort Service — port 30080
```

PHASE 1 — TERRAFORM — Provision devops-master & devops-agent

Use your existing Terraform files to create both CI/CD machines on AWS

You already have 4 Terraform files that provision devops-master and devops-agent on AWS. Below are the exact contents of each file and how to run them.

STEP 1 — Understand Your Terraform Files

provider.tf — AWS Provider Configuration

Configures which AWS account and region to use. Uses role assumption for secure access:

```
provider "aws" {
    region = "us-east-1"
    assume_role {
        role_arn     = "arn:aws:iam::649844050992:role/tfrole"
        session_name = "tfss"
        external_id  = "Dhara"
    }
}
```

data.tf — Fetch the Ubuntu 22.04 AMI

Looks up the exact Ubuntu 22.04 AMI ID to use for the EC2 instances:

```
data "aws_ami" "e2e" {
    filter {
        name   = "name"
        values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-
20260218"]
    }
}
```

locals.tf — Reusable Local Values

Defines instance type and key pair name used by main.tf:

```
locals {
    instance_type = "t3.medium"
    key           = "devops-key"   # Your existing AWS key pair name
}
```

> **IMPORTANT:** Make sure a key pair named 'devops-key' already exists in your AWS account in us-east-1. If your key pair has a different name, update the locals.tf file before running terraform apply.

main.tf — EC2 Instances and Security Group

Creates BOTH devops-master and devops-agent using a for_each loop, plus a shared security group with ports 22, 80, 443, and 8080 open:

```
# Creates two EC2 instances: devops-master and devops-agent
resource "aws_instance" "e2e-ec2" {
    for_each      = toset(["master", "agent"])
    ami           = data.aws_ami.e2e.id
    instance_type = local.instance_type     # t3.medium
    key_name      = local.key               # devops-key
    security_groups = [aws_security_group.e2e-sg.id]
    tags = {
        name = "devops-${each.key}"        # devops-master / devops-agent
    }
}

# Security group allowing all required ports
resource "aws_security_group" "e2e-sg" {
    name        = "ubuntu-sg"
    description = "Allow ports 22, 8080, 80, 443 from Anywhere"

    ingress { from_port = 22,   to_port = 22,   protocol = "tcp", cidr_blocks =
["0.0.0.0/0"], description = "SSH"   }
    ingress { from_port = 80,   to_port = 80,   protocol = "tcp", cidr_blocks =
["0.0.0.0/0"], description = "HTTP"  }
    ingress { from_port = 443,  to_port = 443,  protocol = "tcp", cidr_blocks =
["0.0.0.0/0"], description = "HTTPS" }
    ingress { from_port = 8080, to_port = 8080, protocol = "tcp", cidr_blocks =
["0.0.0.0/0"], description = "App"   }
    egress  { from_port = 0,    to_port = 0,    protocol = "-1",  cidr_blocks =
["0.0.0.0/0"], description = "Allow all" }
}
```

STEP 2 — Set Up Terraform Locally and Run It

Run these commands from your laptop or any machine that has Terraform installed:

```
# Create a folder for Phase 1 Terraform files
mkdir -p ~/devops-project/terraform-phase1
cd ~/devops-project/terraform-phase1

# Place your 4 files here:
#   data.tf  |  locals.tf  |  main.tf  |  provider.tf

# Initialize Terraform — downloads the AWS provider
terraform init
# Expected: Terraform has been successfully initialized!
```

```
# Preview what will be created
terraform plan
# Expected: Plan: 3 to add (2 EC2 instances + 1 security group)

# Create the infrastructure
terraform apply -auto-approve
# Takes 2-3 minutes

# After completion, note the public IPs from AWS Console:
# EC2 → Instances → look for devops-master and devops-agent
# Write down both PUBLIC IP addresses
```

✅ **EXPECTED RESULT:** After terraform apply, go to AWS Console → EC2 → Instances. You will see two instances named devops-master and devops-agent both in running state.

STEP 3 — Verify Both Machines Are Running

```
# SSH into devops-master to confirm it's accessible
# (replace with your actual IP from AWS Console)
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_MASTER_IP>

# You should see:
# ubuntu@ip-172-31-xx-xx:~$

# Exit and also test devops-agent
exit
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_AGENT_IP>
exit
```

PHASE 2 — GIT — Source Code Repository Setup

Create GitHub repo and push all project files

STEP 4 — Create GitHub Repository

1. Go to github.com → sign in → click '+' → New repository
2. Name: devops-project
3. Set to Public — click Create repository
4. Copy the repo URL: https://github.com/<YOUR_USERNAME>/devops-project.git

 IMPORTANT: GitHub no longer accepts your login password for git push. Create a Personal Access Token: GitHub → Settings → Developer Settings → Personal Access Tokens → Tokens (classic) → Generate new token → select 'repo' scope → copy the token. Use this as your password when prompted.

STEP 5 — Install Git and Set Up SSH Key on devops-master

SSH into devops-master and run:

```
# Connect to devops-master
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_MASTER_IP>

# Update system and install Git + Ansible
# These are the ONLY two tools you install manually — everything else is automated
sudo apt update && sudo apt upgrade -y
sudo apt install git ansible -y

# Verify
git --version       # Expected: git version 2.x.x
ansible --version   # Expected: ansible [core 2.x.x]

# Generate SSH key for Ansible to connect to other machines
ssh-keygen -t rsa -b 4096 -f ~/.ssh/id_rsa -N ""

# Copy and save this public key — you will need it in the next step
cat ~/.ssh/id_rsa.pub
# Copy the entire output starting with: ssh-rsa AAAA...
```

STEP 6 — Add Public Key to devops-agent

Open a NEW terminal on your laptop and connect to devops-agent:

```
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_AGENT_IP>

# Create jenkins user
sudo useradd -m -s /bin/bash jenkins 2>/dev/null || echo 'user exists'
```

```
# Set up .ssh directory
sudo mkdir -p /home/jenkins/.ssh
sudo chmod 700 /home/jenkins/.ssh

# Paste the public key you copied from devops-master
echo "PASTE_PUBLIC_KEY_HERE" | sudo tee /home/jenkins/.ssh/authorized_keys

# CRITICAL — set correct permissions
sudo chmod 600 /home/jenkins/.ssh/authorized_keys
sudo chown -R jenkins:jenkins /home/jenkins/.ssh

# Create Jenkins agent working directory
sudo mkdir -p /home/jenkins/agent
sudo chown -R jenkins:jenkins /home/jenkins/agent
exit
```

STEP 7 — Create All Project Files on devops-master

```
# Back on devops-master — create folder structure
mkdir -p ~/devops-project/{app,terraform-phase1,terraform-
phase2,ansible/{playbooks,inventory},kubernetes}
cd ~/devops-project

# Create each file with nano — paste content from the project zip, save with
Ctrl+X → Y → Enter
nano app/index.html
nano Dockerfile
nano Jenkinsfile
nano ansible/inventory/hosts.ini
nano ansible/site.yml
nano ansible/playbooks/setup_master.yml
nano ansible/playbooks/setup_agent.yml
nano ansible/playbooks/configure_jenkins_node.yml
nano ansible/playbooks/setup_kubernetes.yml
nano terraform-phase2/main.tf
nano kubernetes/deployment.yml
nano kubernetes/service.yml

# Also copy your Phase 1 Terraform files into the repo
# Copy data.tf, locals.tf, main.tf, provider.tf into terraform-phase1/
```

STEP 8 — Configure hosts.ini with Your IPs

```
nano ~/devops-project/ansible/inventory/hosts.ini

# Paste this content — replace IPs with your actual values from AWS Console:

[devops_master]
devops-master ansible_host=<DEVOPS_MASTER_IP> ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa
```

```
[devops_agent]
devops-agent  ansible_host=<DEVOPS_AGENT_IP>  ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[k8s_control_plane]
k8s-control-plane ansible_host=<FILL_AFTER_PHASE6> ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[k8s_workers]
k8s-worker-1  ansible_host=<FILL_AFTER_PHASE6>  ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[all:vars]
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

STEP 9 — Push All Files to GitHub

```
cd ~/devops-project
git init
git add .
git commit -m "Initial DevOps project — all files"
git remote add origin https://github.com/<YOUR_USERNAME>/devops-project.git
git branch -M main
git push -u origin main
# Username: your GitHub username
# Password: your Personal Access Token (NOT your GitHub password)
```

✅ **EXPECTED RESULT:** Open github.com/YOUR_USERNAME/devops-project in your browser and confirm all files are visible.

PHASE 3 — ANSIBLE — Configure devops-master and devops-agent

Automatically install Jenkins, Terraform, Docker using Ansible playbooks

Ansible reads your hosts.ini file and automatically installs all required tools on each machine. You just run one command per playbook.

STEP 10 — Test Ansible Can Reach Both Machines

```
cd ~/devops-project

ansible all -i ansible/inventory/hosts.ini -m ping

# Expected:
# devops-master | SUCCESS => { "ping": "pong" }
# devops-agent  | SUCCESS => { "ping": "pong" }
```

IMPORTANT: If you see UNREACHABLE — check that the IP in hosts.ini is correct and security group allows port 22.

STEP 11 — Run setup_master.yml — Install Jenkins + Terraform + Java on devops-master

```
ansible-playbook -i ansible/inventory/hosts.ini \
  ansible/playbooks/setup_master.yml

# Takes 5-10 minutes. This installs:
#   Java 17, Jenkins, Terraform on devops-master

# At the end — look for and COPY this output:
# TASK [Display Jenkins initial admin password]
# msg: Jenkins Initial Admin Password: xxxxxxxxxxxxxxxxxxxxxxxx

# SAVE THAT PASSWORD — needed in Phase 4
```

✕ **ERROR FIX:** GPG error for Jenkins? Run this fix on devops-master then re-run the playbook: sudo rm -f /usr/share/keyrings/jenkins-keyring.asc /etc/apt/sources.list.d/jenkins.list curl -fsSL https://pkg.jenkins.io/debian-stable/jenkins.io-2023.key | sudo gpg --dearmor -o /usr/share/keyrings/jenkins-keyring.gpg echo "deb [signed-by=/usr/share/keyrings/jenkins-keyring.gpg] https://pkg.jenkins.io/debian-stable binary/" | sudo tee /etc/apt/sources.list.d/jenkins.list sudo apt update

STEP 12 — Run setup_agent.yml — Install Docker + Java on devops-agent

```
ansible-playbook -i ansible/inventory/hosts.ini \
  ansible/playbooks/setup_agent.yml

# Takes 3-5 minutes. This installs:
#   Java 17, Docker CE on devops-agent

# Expected at end:
# PLAY RECAP
# devops-agent : ok=12  changed=8  unreachable=0  failed=0
```

✕ **ERROR FIX:** Docker GPG error? Run on devops-master then re-run playbook: ansible devops_agent -i ansible/inventory/hosts.ini -b -m shell -a 'rm -f /usr/share/keyrings/docker-archive-keyring.gpg /etc/apt/sources.list.d/docker.list && install -m 0755 -d /etc/apt/keyrings && curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg && chmod a+r /etc/apt/keyrings/docker.gpg'

STEP 13 — Verify Both Machines Are Configured

```
# Check Jenkins is running on devops-master
ansible devops_master -i ansible/inventory/hosts.ini -b -m shell \
  -a 'systemctl status jenkins --no-pager | head -3'
# Expected: Active: active (running)

# Check Docker on devops-agent
ansible devops_agent -i ansible/inventory/hosts.ini -b -m shell \
  -a 'docker --version'
# Expected: Docker version 26.x.x

# Check Terraform on devops-master
ansible devops_master -i ansible/inventory/hosts.ini -b -m shell \
  -a 'terraform version'
# Expected: Terraform v1.x.x
```

PHASE 4 — JENKINS — CI/CD Pipeline Setup

Configure Jenkins UI, connect devops-agent, run the pipeline

STEP 14 — Open Jenkins in Your Browser

On your laptop open a browser and go to:

```
http://<DEVOPS_MASTER_IP>:8080
```

5. Paste the admin password saved from Step 11
6. Click 'Install suggested plugins' — wait 3-5 minutes
7. Create admin user — fill username, password, email → Save and Continue
8. Jenkins URL — leave as default → Save and Finish → Start using Jenkins

STEP 15 — Install SSH Build Agents Plugin

9. Manage Jenkins → Plugins → Available plugins tab
10. Search: SSH Build Agents → check the box → Install
11. Check 'Restart Jenkins after installation' → wait for restart → log back in

STEP 16 — Add SSH Private Key Credential

Get the private key from devops-master:

```
# On devops-master
sudo cat /var/lib/jenkins/.ssh/id_rsa
# Copy EVERYTHING including:
# -----BEGIN RSA PRIVATE KEY-----
# ...all lines...
# -----END RSA PRIVATE KEY-----
```

In Jenkins UI:

12. Manage Jenkins → Credentials → System → Global credentials → Add Credentials
13. Kind: SSH Username with private key
14. ID: jenkins-agent-ssh
15. Username: jenkins
16. Private Key: Enter directly → Add → paste the entire private key → Create

STEP 17 — Add devops-agent as Jenkins Node

17. Manage Jenkins → Nodes → New Node

18. Node name: devops-agent → Permanent Agent → OK

Field	Value to Enter
`Remote root directory`	/home/jenkins/agent
`Labels`	devops-agent docker
`Launch method`	Launch agents via SSH
`Host`	<DEVOPS_AGENT_IP>
`Credentials`	select: jenkins-agent-ssh
`Host Key Verification`	Non verifying Verification Strategy

19. Click Save → click on devops-agent → click 'Launch agent'
20. Wait 30 seconds — status dot must turn GREEN ✓

> ✗ **ERROR FIX:** If authentication fails: verify the jenkins user exists on devops-agent (run: id jenkins), check /home/jenkins/.ssh/authorized_keys has the public key, and permissions are 700 for .ssh and 600 for authorized_keys.

STEP 18 — Create Jenkins CI/CD Pipeline Job

21. Jenkins Dashboard → New Item
22. Name: prt-devops-pipeline → Pipeline → OK
23. Scroll to Pipeline section → Definition: Pipeline script from SCM
24. SCM: Git → Repository URL: https://github.com/<YOUR_USERNAME>/devops-project.git
25. Branch: */main → Script Path: Jenkinsfile → Save

STEP 19 — Run the Pipeline and Watch It Execute

26. Click 'Build Now' → click build #1 → click Console Output

Stage	What Happens
`1 — Checkout`	Pulls latest code from your GitHub repository onto devops-agent
`2 — Build Docker Image`	Runs docker build using your Dockerfile — creates prt-devops-app:latest
`3 — Verify Image`	Runs docker images to confirm the image exists
`4 — Smoke Test`	Starts container on port 8099, sends curl request, stops container

> ✓ **EXPECTED RESULT:** Pipeline completed successfully! Docker image prt-devops-app:latest is built and ready on devops-agent.

PHASE 5 — DOCKER — What Was Built by Jenkins

Review Dockerfile and the container image

The Dockerfile is stored in Git and used by Jenkins during the pipeline. Here is what it contains and what it does:

STEP 20 — Dockerfile — NGINX Container Definition

```
# Dockerfile — stored in root of devops-project/

FROM nginx:stable-alpine

LABEL maintainer="devops-team"
LABEL description="PRT DevOps Pipeline - NGINX App"

# Remove default NGINX page
RUN rm -rf /usr/share/nginx/html/*

# Copy our application HTML page
COPY app/index.html /usr/share/nginx/html/index.html

# Expose port 80
EXPOSE 80

# Start NGINX
CMD ["nginx", "-g", "daemon off;"]
```

STEP 21 — Jenkinsfile — CI/CD Pipeline Definition

```
// Jenkinsfile — stored in root of devops-project/

pipeline {
    agent { label 'devops-agent' }   // Runs on devops-agent which has Docker

    environment {
        IMAGE_NAME = "prt-devops-app"
        IMAGE_TAG  = "latest"
    }

    stages {
        stage('Checkout') {
            steps { checkout scm }   // Pull code from GitHub
        }
        stage('Build Docker Image') {
            steps {
                sh 'docker build -t ${IMAGE_NAME}:${IMAGE_TAG} .'
            }
        }
        stage('Verify Image') {
            steps {
                sh 'docker images | grep ${IMAGE_NAME}'
```

```
            }
        }
        stage('Smoke Test') {
            steps {
                sh '''
                    docker run -d --name smoke-test -p 8099:80
${IMAGE_NAME}:${IMAGE_TAG}
                    sleep 3
                    curl -f http://localhost:8099 || exit 1
                    docker stop smoke-test && docker rm smoke-test
                '''
            }
        }
    }
    post {
        always { sh 'docker rm -f smoke-test 2>/dev/null || true' }
    }
}
```

STEP 22 — Verify Docker Image on devops-agent

```
# SSH into devops-agent
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_AGENT_IP>

# List Docker images
docker images
# Expected:
# REPOSITORY        TAG       IMAGE ID       SIZE
# prt-devops-app    latest    xxxxxxxxxxxx   ~25MB

# Optionally run it locally to test
docker run -d -p 8080:80 prt-devops-app:latest
curl http://localhost:8080
# Expected: HTML with 'Congratulations! You are halfway done with the PRT.'
```

PHASE 6 — TERRAFORM — Provision Kubernetes Machines

Use Terraform from devops-master to create k8s-control-plane and k8s-worker-1

Just like Phase 1 used Terraform to create devops-master and devops-agent, this phase uses Terraform from devops-master to create the two Kubernetes machines.

STEP 23 — Configure AWS Credentials on devops-master

```
# SSH into devops-master
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_MASTER_IP>

# Set AWS credentials
# Get from: AWS Console → IAM → Users → Security credentials → Create access key
export AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY"
export AWS_SECRET_ACCESS_KEY="YOUR_SECRET_KEY"
export AWS_DEFAULT_REGION="us-east-1"

# Make permanent across sessions
echo 'export AWS_ACCESS_KEY_ID="YOUR_ACCESS_KEY"'     >> ~/.bashrc
echo 'export AWS_SECRET_ACCESS_KEY="YOUR_SECRET_KEY"' >> ~/.bashrc
echo 'export AWS_DEFAULT_REGION="us-east-1"'          >> ~/.bashrc
source ~/.bashrc
```

STEP 24 — The Terraform Config for K8s Machines (terraform-phase2/main.tf)

This file is already in your Git repo under terraform-phase2/. It creates k8s-control-plane and k8s-worker-1 with port 30080 added for Kubernetes NodePort access:

```
# terraform-phase2/main.tf

terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = { source = "hashicorp/aws", version = "~> 5.0" }
  }
}

provider "aws" {
  region = "us-east-1"
}

locals {
  instance_type = "t3.medium"
  key           = "devops-key"   # same key pair as Phase 1
}

data "aws_ami" "ubuntu" {
```

```
  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-20260218"]
  }
}

resource "aws_security_group" "k8s-sg" {
  name        = "k8s-cluster-sg"
  description = "Security group for Kubernetes nodes"
  ingress { from_port = 22,    to_port = 22,    protocol = "tcp", cidr_blocks =
["0.0.0.0/0"] }
  ingress { from_port = 6443,  to_port = 6443,  protocol = "tcp", cidr_blocks =
["0.0.0.0/0"] }
  ingress { from_port = 30080, to_port = 30080, protocol = "tcp", cidr_blocks =
["0.0.0.0/0"] }
  ingress { from_port = 10250, to_port = 10260, protocol = "tcp", self = true }
  ingress { from_port = 2379,  to_port = 2380,  protocol = "tcp", self = true }
  egress  { from_port = 0,     to_port = 0,     protocol = "-1",  cidr_blocks =
["0.0.0.0/0"] }
}

resource "aws_instance" "k8s-control-plane" {
  ami             = data.aws_ami.ubuntu.id
  instance_type   = local.instance_type
  key_name        = local.key
  security_groups = [aws_security_group.k8s-sg.id]
  tags = { Name = "k8s-control-plane" }
}

resource "aws_instance" "k8s-worker-1" {
  ami             = data.aws_ami.ubuntu.id
  instance_type   = local.instance_type
  key_name        = local.key
  security_groups = [aws_security_group.k8s-sg.id]
  tags = { Name = "k8s-worker-1" }
}

output "k8s_control_plane_ip" { value = aws_instance.k8s-control-plane.public_ip }
output "k8s_worker_1_ip"      { value = aws_instance.k8s-worker-1.public_ip }
```

STEP 25 — Run Terraform from devops-master

```
cd ~/devops-project/terraform-phase2

terraform init
# Expected: Terraform has been successfully initialized!

terraform plan
# Expected: Plan: 3 to add

terraform apply -auto-approve
# Takes 2-3 minutes

# Outputs at end:
# k8s_control_plane_ip = "54.x.x.x"
```

```
# k8s_worker_1_ip      = "52.x.x.x"

# WRITE DOWN BOTH IPs — needed in next step
```

STEP 26 — Update hosts.ini with K8s IPs

```
nano ~/devops-project/ansible/inventory/hosts.ini

# Update k8s sections with actual IPs from terraform output:
[k8s_control_plane]
k8s-control-plane ansible_host=54.x.x.x ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

[k8s_workers]
k8s-worker-1  ansible_host=52.x.x.x ansible_user=ubuntu
ansible_ssh_private_key_file=~/.ssh/id_rsa

# Save → Ctrl+X → Y → Enter

# Accept host keys for the new machines
ssh-keyscan -H 54.x.x.x 52.x.x.x >> ~/.ssh/known_hosts
```

PHASE 7 — KUBERNETES — Cluster Setup

Use Ansible to install K8s, initialize cluster, join worker node

STEP 27 — Run the Kubernetes Setup Playbook

```
cd ~/devops-project

ansible-playbook -i ansible/inventory/hosts.ini \
  ansible/playbooks/setup_kubernetes.yml

# Takes 10-15 minutes. Three plays run automatically:

# PLAY 1 — Common setup on ALL K8s nodes:
#   disables swap, loads kernel modules, sets sysctl params
#   installs containerd, kubeadm, kubelet, kubectl

# PLAY 2 — Initialize control plane:
#   runs kubeadm init --pod-network-cidr=10.244.0.0/16
#   sets up kubectl for ubuntu user
#   installs Flannel CNI network plugin
#   generates join command for worker

# PLAY 3 — Join worker to cluster:
#   runs kubeadm join on k8s-worker-1
```

✕ **ERROR FIX:** GPG error for Docker/Kubernetes repos during this playbook? Remove old keys and re-add using: curl -fsSL <KEY_URL> | gpg --dearmor -o /etc/apt/keyrings/<name>.gpg See Troubleshooting section at end of this document.

✕ **ERROR FIX:** kubeadm init fails with CRI error? Fix containerd on both K8s nodes: ansible k8s_control_plane:k8s_workers -i ansible/inventory/hosts.ini -b -m shell -a 'systemctl stop containerd && rm -f /etc/containerd/config.toml && containerd config default | tee /etc/containerd/config.toml && sed -i "s/SystemdCgroup = false/SystemdCgroup = true/g" /etc/containerd/config.toml && systemctl restart containerd' Then re-run the playbook.

STEP 28 — Verify Kubernetes Cluster is Ready

```
# SSH into k8s-control-plane
ssh -i ~/Downloads/devops-key.pem ubuntu@<K8S_MASTER_IP>

kubectl get nodes

# Expected — both nodes MUST show Ready:
# NAME                STATUS   ROLES           AGE   VERSION
# k8s-control-plane   Ready    control-plane   3m    v1.29.x
# k8s-worker-1        Ready    <none>          2m    v1.29.x

# Check all system pods are running
kubectl get pods -n kube-system
```

```
# All pods should be Running or Completed
```

✅ **EXPECTED RESULT:** Both nodes show Ready — Kubernetes cluster is fully operational!

PHASE 8 — KUBERNETES — Deploy the Application

Transfer Docker image and deploy NGINX app with 2 replicas

STEP 29 — Transfer Docker Image from devops-agent to K8s Nodes

```
# On devops-agent — export the Docker image
ssh -i ~/Downloads/devops-key.pem ubuntu@<DEVOPS_AGENT_IP>

docker save prt-devops-app:latest | gzip > /tmp/prt-devops-app.tar.gz
ls -lh /tmp/prt-devops-app.tar.gz   # Should be ~25-50MB

# Copy to k8s-control-plane
scp -i ~/Downloads/devops-key.pem /tmp/prt-devops-app.tar.gz
ubuntu@<K8S_MASTER_IP>:~

# Copy to k8s-worker-1
scp -i ~/Downloads/devops-key.pem /tmp/prt-devops-app.tar.gz
ubuntu@<K8S_WORKER_IP>:~
exit
```

STEP 30 — Load Image on Both K8s Nodes

```
# Load on k8s-control-plane
ssh -i ~/Downloads/devops-key.pem ubuntu@<K8S_MASTER_IP>
sudo ctr -n k8s.io images import ~/prt-devops-app.tar.gz
sudo ctr -n k8s.io images list | grep prt-devops-app   # Verify
exit

# Load on k8s-worker-1
ssh -i ~/Downloads/devops-key.pem ubuntu@<K8S_WORKER_IP>
sudo ctr -n k8s.io images import ~/prt-devops-app.tar.gz
sudo ctr -n k8s.io images list | grep prt-devops-app   # Verify
exit
```

STEP 31 — Kubernetes Manifest Files

deployment.yml — Creates 2 Running Pods

```
# kubernetes/deployment.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: prt-devops-app
  namespace: default
spec:
  replicas: 2
  selector:
    matchLabels:
      app: prt-devops-app
  template:
```

```
    metadata:
      labels:
        app: prt-devops-app
    spec:
      containers:
        - name: prt-devops-app
          image: prt-devops-app:latest
          imagePullPolicy: IfNotPresent   # Uses local image — no registry needed
          ports:
            - containerPort: 80
```

service.yml — Exposes App on Port 30080

```
# kubernetes/service.yml
apiVersion: v1
kind: Service
metadata:
  name: prt-devops-app-service
  namespace: default
spec:
  type: NodePort
  selector:
    app: prt-devops-app
  ports:
    - protocol: TCP
      port: 80          # cluster-internal port
      targetPort: 80    # container port
      nodePort: 30080   # external access port
```

STEP 32 — Deploy the Application

```
# On k8s-control-plane
ssh -i ~/Downloads/devops-key.pem ubuntu@<K8S_MASTER_IP>

# Clone repo
git clone https://github.com/<YOUR_USERNAME>/devops-project.git
cd devops-project

# Deploy
kubectl apply -f kubernetes/deployment.yml
# Expected: deployment.apps/prt-devops-app created

kubectl apply -f kubernetes/service.yml
# Expected: service/prt-devops-app-service created
```

STEP 33 — Verify Everything is Running

```
# Check pods are running
kubectl get pods -o wide
# Expected:
```

```
# NAME                            READY   STATUS    NODE
# prt-devops-app-xxxxxxx-aaaaa    1/1     Running   k8s-worker-1
# prt-devops-app-xxxxxxx-bbbbb    1/1     Running   k8s-worker-1

# Check service
kubectl get service prt-devops-app-service
# Expected:
# NAME                    TYPE       PORT(S)        AGE
# prt-devops-app-service  NodePort   80:30080/TCP   30s

# Test from inside cluster
curl http://localhost:30080
# Expected: HTML page content
```

STEP 34 — Access the Application in Your Browser

```
# Open in browser:
http://<K8S_WORKER_IP>:30080

# You will see:
# Congratulations! You are halfway done with the PRT. Keep going.
```

✅ **EXPECTED RESULT:** Application is live at http://<K8S_WORKER_IP>:30080 — Full end-to-end pipeline complete!

Final Verification Checklist

#	What to Verify	Command / Where to Check	Expected Result
1	devops-master running	AWS Console → EC2 → Instances	Running state
2	devops-agent running	AWS Console → EC2 → Instances	Running state
3	Ansible reaches all nodes	ansible all -i hosts.ini -m ping	pong from all
4	Jenkins installed	curl http://<master>:8080	HTTP 200 or 403
5	Docker on agent	docker --version (on devops-agent)	Docker 26.x.x
6	Terraform on master	terraform version (on devops-master)	Terraform v1.x.x
7	Jenkins agent connected	Manage Jenkins → Nodes → devops-agent	Green dot
8	Jenkins pipeline success	Jenkins → prt-devops-pipeline → Build #1	Finished: SUCCESS
9	Docker image built	docker images (on devops-agent)	prt-devops-app listed
10	K8s nodes Ready	kubectl get nodes	Both show Ready
11	K8s pods running	kubectl get pods	2/2 Running
12	K8s service created	kubectl get service	NodePort 30080
13	App accessible in browser	http://<K8S_WORKER_IP>:30080	Congratulations page

Troubleshooting Guide

Error 1 — GPG Key Error (any repo: Docker, Jenkins, Kubernetes)

Symptom: 'NO_PUBKEY' or 'repository is not signed' during any Ansible playbook

```
# Universal fix — use this pattern for any GPG failure

# Remove old broken key and repo
sudo rm -f /usr/share/keyrings/<name>.gpg
sudo rm -f /etc/apt/sources.list.d/<name>.list

# Create keyrings directory
sudo install -m 0755 -d /etc/apt/keyrings

# Download key correctly with curl + gpg --dearmor
curl -fsSL <KEY_URL> | sudo gpg --dearmor -o /etc/apt/keyrings/<name>.gpg
sudo chmod a+r /etc/apt/keyrings/<name>.gpg

# Add repo referencing the new key
echo "deb [signed-by=/etc/apt/keyrings/<name>.gpg] <REPO_URL>" | \
  sudo tee /etc/apt/sources.list.d/<name>.list
sudo apt update

# Re-run the playbook — it skips already completed tasks
```

Error 2 — Jenkins Agent SSH Authentication Failed

Symptom: 'Server rejected the private key' in Jenkins node log

```
# Check 1 — jenkins user exists on devops-agent
ssh ubuntu@<DEVOPS_AGENT_IP>
id jenkins
# Expected: uid=xxxx(jenkins) groups=xxxx(jenkins)

# Check 2 — authorized_keys has the public key
sudo cat /home/jenkins/.ssh/authorized_keys
# Must show one line starting with ssh-rsa...

# Check 3 — fix permissions if wrong
sudo chmod 700 /home/jenkins/.ssh
sudo chmod 600 /home/jenkins/.ssh/authorized_keys
sudo chown -R jenkins:jenkins /home/jenkins/.ssh

# Check 4 — test SSH manually from devops-master
sudo su - jenkins
ssh -i ~/.ssh/id_rsa -o StrictHostKeyChecking=no jenkins@<DEVOPS_AGENT_IP>
# Must connect without password prompt
```

Error 3 — kubeadm init Fails with CRI / containerd Error

Symptom: 'container runtime is not running' or 'CRI v1 runtime API Unimplemented'

```
# Run on BOTH K8s nodes
sudo systemctl stop containerd
sudo rm -f /etc/containerd/config.toml
sudo containerd config default | sudo tee /etc/containerd/config.toml
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/g'
/etc/containerd/config.toml

# Verify
grep 'SystemdCgroup' /etc/containerd/config.toml
# Must show: SystemdCgroup = true

sudo systemctl restart containerd
sudo systemctl status containerd
# Must show: Active: active (running)

# Reset partial kubeadm state on control-plane
sudo kubeadm reset -f

# Re-run the playbook or run kubeadm init manually
sudo kubeadm init --pod-network-cidr=10.244.0.0/16 --apiserver-advertise-
address=<K8S_MASTER_PRIVATE_IP>
```

Error 4 — SSH Fingerprint Warning (yes/no prompt)

Symptom: 'The authenticity of host cannot be established'

```
# This is NOT an error — type 'yes' and press Enter

# To prevent permanently — accept all host keys at once:
ssh-keyscan -H <IP1> <IP2> <IP3> <IP4> >> ~/.ssh/known_hosts

# Or add to ansible config:
echo 'host_key_checking = False' >> ~/.ansible.cfg
```

Error 5 — K8s Nodes Stuck in NotReady State

```
# Re-apply Flannel CNI
kubectl apply -f https://raw.githubusercontent.com/flannel-
io/flannel/master/Documentation/kube-flannel.yml

# Wait 60 seconds then check
kubectl get nodes

# Check flannel pods
kubectl get pods -n kube-flannel
# All flannel pods must show Running
```

Error 6 — Pods Show ImagePullBackOff

```
# The image was not imported on the worker node
ssh ubuntu@<K8S_WORKER_IP>
sudo ctr -n k8s.io images list | grep prt-devops-app

# If missing — re-import
sudo ctr -n k8s.io images import ~/prt-devops-app.tar.gz

# Restart the deployment
kubectl rollout restart deployment/prt-devops-app
```

Error 7 — App Not Accessible on Port 30080

```
# Check 1 — AWS Security Group for k8s-worker-1
# AWS Console → EC2 → k8s-worker-1 → Security tab → Edit inbound rules
# Add: Custom TCP, Port 30080, Source 0.0.0.0/0

# Check 2 — pods are running
kubectl get pods
# Must show 1/1 Running

# Check 3 — service exists
kubectl get service
# Must show NodePort 80:30080

# Check 4 — test from inside cluster
curl http://<K8S_WORKER_PRIVATE_IP>:30080
# If this works but browser fails — it is a security group issue
```